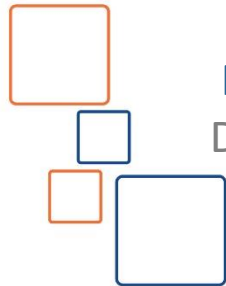


CORE JAVA PROGRAMMING

AN INTRODUCTION TO JAVA



Presented By
Dr. Eman Hesham, DBA.



Java™ Education
and Technology Services



Invest In Yourself,
Develop Your Career

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces**
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Lesson 5

Modifiers, Access Specifiers, Packages and Interfaces



Modifiers and Access Specifiers

- ▶ Modifiers-**How**- and Access Specifiers -**Who**- are a set of keywords that affect the way we work with features (classes, methods, and variables).
- ▶ The following table illustrates these keywords and how they are used.

Modifiers and Access Specifiers cont'd

Keyword	Top Level Class	Methods	Variables	Free Floating Block
public	Yes	Yes	Yes	-
protected	-	Yes	Yes	-
(friendly)*	Yes	Yes	Yes	-
private	-	Yes	Yes	-
final	Yes	Yes	Yes	-
static	-	Yes	Yes	Yes
abstract	Yes	Yes	-	-



Packages



Packages

- ▶ In the preceding lessons, the name of each example class was taken from the same name space. This means that a unique name had to be used for each class to avoid name collisions.
- ▶ After a while, without some way to manage the name space, you could run out of convenient, descriptive names for individual classes.
- ▶ You also need some way to be assured that the name you choose for a class will be reasonably unique and not collide with class names chosen by other programmers.

Packages

- ▶ Java provides a mechanism for partitioning the class name space into more manageable chunks.
- ▶ This mechanism is the package. The package is both a naming and a visibility control mechanism.
- ▶ You can define classes inside a package that are not accessible by code outside that package.
- ▶ You can also define class members that are exposed only to other members of the same package.

Defining a Packages

- ▶ To create a package is quite easy: simply include a **package** command as the first statement in a Java source file.
- ▶ Any classes declared within that file will belong to the specified package.
- ▶ The **package** statement defines a name space in which classes are stored. If you omit the **package** statement, the class names are put into the default package, which has no name.
- ▶ This is the general form of the **package** statement:

pkg is the name of the package;

```
package pkg;
```

Defining a Packages

- ▶ example, the following statement creates a package called **mypackage**:

```
package mypackage;
```

- ▶ Java uses file system directories to store packages.
- ▶ For example, the **.class** files for any classes you declare to be part of **mypackage** must be stored in a directory called **mypackage**.
- ▶ The general form of a multileveled package statement is shown here:

```
package pkg1[.pkg2[.pkg3]];  package a.b.c;
```



Packages and Member Access

- ▶ Packages add another dimension to access control. As you will see, Java provides many levels of protection to allow fine-grained control over the visibility of variables and methods within classes, subclasses, and packages.
- ▶ Classes and packages are both means of encapsulating and containing the name space and scope of variables and methods.
- ▶ Packages act as containers for classes and other subordinate packages. Classes act as containers for data and code.

Packages and Member Access

- ▶ Java addresses four categories of visibility for class members:
 - ▶ Subclasses in the same package
 - ▶ Non-subclasses in the same package
 - ▶ Subclasses in different packages
 - ▶ Classes that are neither in the same package nor subclasses
- ▶ The three access modifiers, **private**, **public**, and **protected**, provide a variety of ways to produce the many levels of access required by these categories.
- ▶ The following table sums up the interactions.

Packages and Member Access

	private	No Modifier	protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Importing packages

- ▶ In a Java source file, **import** statements occur immediately following the **package** statement (if it exists) and before any class definitions. This is the general form of the **import** statement:

```
import pkg1 [.pkg2].(classname | *);
```

Example:

```
package pkg1.pkg2;  
import java.util.Date;  
import java.io.*;
```

Standard Java SE packages

- ▶ All of the standard Java SE classes included with Java begin with the name **java**.
- ▶ The basic language functions are stored in a package called **java.lang**.
- ▶ Normally, you have to import every package or class that you want to use, but since Java is useless without much of the functionality in **java.lang**, it is implicitly imported by the compiler for all programs.

Standard Java SE packages

Package Name	Description
java.lang	Contains language support classes (for e.g classes which defines primitive data types, math operations, etc.) . This package is automatically imported.
java.io	Contains classes for supporting input / output operations.
java.util	Contains utility classes which implement data structures like Linked List, Hash Table, Dictionary, etc and support for Date / Time operations.
java.awt	Contains classes for implementing the components of graphical user interface (like buttons, menus, etc.).
java.net	Contains classes for supporting networking operations.

Interfaces

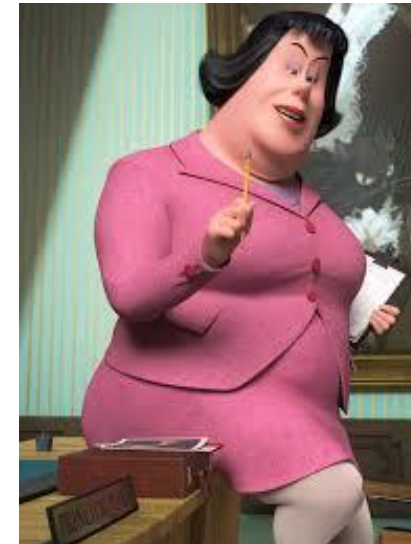
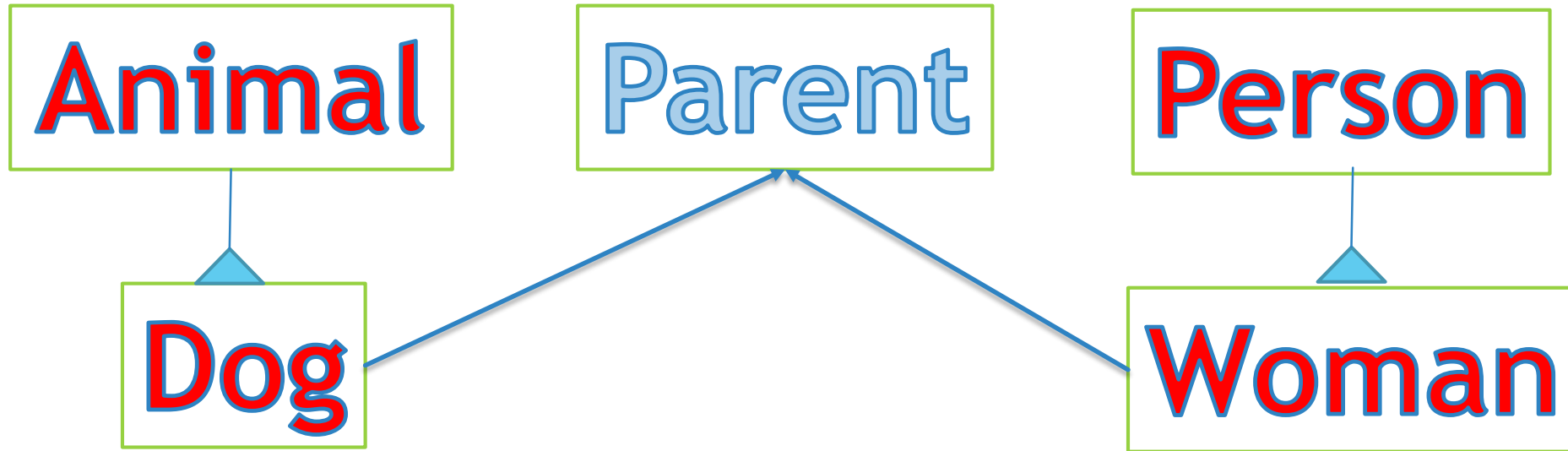
Interfaces

- ▶ In OOP, it is sometimes helpful to define what a class must do but not how it will do it.
- ▶ An **abstract method** defines the **signature** for a method but provides **no implementation**.
- ▶ A **subclass** must provide its own **implementation** of each abstract method defined by its **superclass**.
- ▶ Thus, an **abstract method** specifies the **interface** to the method but not the *implementation*.
- ▶ In Java, you can fully separate a class' interface from its implementation by using the keyword **interface**.

Interfaces

- ▶ An *interface* is syntactically similar to an **abstract class**, in that you can specify one or more methods that have no body.
- ▶ Those methods must be **implemented by a class** in order for their actions to be defined.
- ▶ An *interface* specifies what **must be done**, but **not how to do it**.
- ▶ Once an interface is defined, any number of classes can implement it.
- ▶ Also, one class can implement any number of interfaces.

Interfaces



Interfaces

Here is an example of an **interface** definition.

```
public interface Numbers
{
    int getNext(); // return next number in series
    void reset(); // restart
    void setStart(int x); // set starting value
}
```

Implementing Interfaces

The general form of a class that includes the **implements** clause looks like this:

```
Access Specifier class classname extends superclass implements interface {  
    // class-body  
}
```

Implementing Interfaces

```
// Implement Numbers.  
class ByTwos implements Numbers  
{  
    int start;  
    int val;  
    Public ByTwos() {  
        start = 0;  
        val = 0;  
    }  
    public int getNext() {  
        val += 2;  
        return val;  
    }  
    public void reset() {  
        val = start;  
    }  
    public void setStart(int x) {  
        start = x;  
        val = x;  
    }  
}
```

- Class **ByTwos** implements the **Numbers** interface
- Notice that the methods `getNext( )`, `reset( )`, and `setStart( )` are declared using the public access specifier

Implementing Interfaces

```
// Implement Numbers.  
class ByThrees implements Numbers  
{  
    int start;  
    int val;  
    public ByThrees() {  
        start = 0;  
        val = 0;  
    }  
    public int getNext() {  
        val += 3; return val;  
    }  
    public void reset() {  
        val = start;  
    }  
    public void setStart (int x) {  
        start = x;  
        val = x;  
    }  
}
```

- Class **ByThrees** provides another implementation of the **Numbers** interface
- Notice that the methods `getNext()`, `reset()`, and `setStart()` are declared using the public access specifier

Using interface reference

```
class Demo2
{
    public static void main (String args[])
    {
        ByTwos twoOb = new ByTwos();
        ByThrees threeOb = new ByThrees();
        Numbers ob;
        for(int i=0; i < 5; i++) {
            ob = twoOb;
            System.out.println("Next ByTwos value is " + ob.getNext());
            ob = threeOb;
            System.out.println("Next ByThrees value is " + ob.getNext());
        }
    }
}
```

General Consideration about interfaces

- ▶ **Variables** can be declared in an interface, but they are implicitly **public**, **static**, and **final**.
- ▶ To define a set of shared constants, create an interface that contains only these constants, without any methods.
- ▶ One **interface** can **inherit** another by use of the keyword **extends**. *The syntax is the same as for inheriting classes.*
- ▶ When a class implements an interface that inherits another interface, it must provide implementations for all methods required by the interface inheritance chain.

Default Methods

- ▶ Prior to JDK 8, an interface **could not** define any implementation whatsoever.
- ▶ The release of JDK 8 changed this by adding a new capability to interface called the default method.
- ▶ A default method lets you define a **default implementation** for an interface method.
- ▶ You specify an interface default method by using the **default** keyword

Default Methods

```
interface InterfaceA {  
  
    public void saySomething();  
  
    default public void sayHi() {  
        System.out.println("Hi");  
    }  
  
}
```

```
public class MyClass implements InterfaceA  
{  
  
    public void saySomething() {  
        System.out.println("Hello World");  
    }  
  
}
```

Default Methods

► Extending Interfaces That Contain Default Methods

- Not mention the default method at all, which lets your extended interface inherit the default method.

```
interface Intf1 {  
    default void method() { doSomething(); }  
}  
  
interface Intf2 extends Intf1 {  
  
}
```

Default Methods

► Extending Interfaces That Contain Default Methods

- Redefine the default method, which overrides it.

```
interface Intf1 {  
    default void method() { doSomething(); }  
}  
  
interface Intf2 extends Intf1 {  
    default void method() { doAnother(); }  
}
```

Default Methods

► Extending Interfaces That Contain Default Methods

- Re-declare the default method, which makes it abstract.

```
interface Intf1 {  
    default void method() { doSomething(); }  
}  
  
interface Intf2 extends Intf1 {  
    abstract void method();  
}
```


Use static Methods in an Interface

- ▶ JDK 8 added another new capability to interface:
 - ▶ the ability to define one or more **static methods**.
- ▶ Like static methods in a class, a static method defined by an interface can be called independently of any **object**.
- ▶ Thus, no implementation of the interface is necessary, and no instance of the interface is required in order to call a static method.

Use static Methods in an Interface

```
public interface MyIF {  
    // This is a "normal" interface method declaration.  
    //It does NOT define a default implementation.  
    int x=10;  
    int getUserID();  
  
    // This is a default method. Notice that it provides a  
    //default implementation.  
    default int getAdminID()  
    {  
        return 1;  
    }  
  
    // This is a static interface method.  
    static int getUniversalID()  
    {  
        return 0;  
    }  
}
```

Private Interface Methods

- ▶ Beginning with JDK 9, an interface can include a private method.
- ▶ A private interface method can be called only by a default method or another private method defined by the same interface.
- ▶ Because a private interface method is specified **private**, it cannot be used by code outside the interface in which it is defined.
- ▶ This restriction includes sub-interface because a private interface method is not inherited by a sub-interface.

Private Interface Methods

- ▶ The key benefit of a private interface method is that it lets two or more default methods use a common piece of code, thus avoiding code duplication.
- ▶ Consider an example of a stack of integers, let us define an interface called *IntStack* that has two default methods called *popNElements()* and *skipAndPopNElements()*.
- ▶ The first returns an array that contains the top *N* elements on the stack. The second skips a specified number of elements and then returns an array that contains the next *N* elements.

Private Interface Methods

- ▶ Both use a private method called `getElements( )` to obtain an array of the specified number of elements from the stack.

```
//Another version of IntStack that has a private interface
//method that is used by two default methods.
interface IntStack {
    void push(int item); // store an item
    int pop(); // retrieve an item
    // A default method that returns an array that contains
    // the top n elements on the stack.
    default int[] popNElements(int n) {
        // Return the requested elements.
        return getElements(n);
    }
}
```

Private Interface Methods

```
// A default method that returns an array that contains
// the next n elements on the stack after skipping
elements.
    default int[] skipAndPopNElements(int skip, int n) {
        // Skip the specified number of elements.
        getElements(skip);
        // Return the requested elements.
        return getElements(n);
    }
// A private method that returns an array containing
// the top n elements on the stack
    private int[] getElements(int n) {
        int[] elements = new int[n];
        for (int i = 0; i < n; i++)
            elements[i] = pop();
        return elements;
    }
}
```

Private Interface Methods

- ▶ Notice that both `popNElements( )` and `skipAndPopNElements( )` use the private `getElements( )` method to obtain the array to return.
- ▶ This prevents both methods from having to duplicate the same code sequence.
- ▶ Keep in mind that because `getElements( )` is private, it cannot be called by code outside `IntStack`. Thus, its use is limited to the default methods inside `IntStack`.

Private Interface Methods

- ▶ Also, because `getElements( )` uses the `pop( )` method to obtain stack elements, it will automatically call the implementation of `pop( )` provided by the `IntStack` implementation.
- ▶ Thus, `getElements( )` will work for any stack class that implements `IntStack`.

Interfaces and Callbacks

- ▶ A common pattern in programming is the *callback* pattern.
- ▶ In this pattern, you specify the action that should occur whenever a particular event happens.
- ▶ Consider the following example:
- ▶ The *javax.swing* package contains a `Timer` class that is useful if you want to be notified whenever a time interval has elapsed.
- ▶ For example, if a part of your program contains a clock, you can ask to be notified every second so that you can update the clock face.

Interfaces and Callbacks

- ▶ When you construct a timer, you set the time interval and you tell it what it should do whenever the time interval has elapsed.
- ▶ How do you tell the timer what it should do?
- ▶ You pass an object of some class. The timer then calls one of the methods on that object.
- ▶ Of course, the timer needs to know what method to call. The timer requires that you specify an object of a class that implements the ***ActionListener*** interface of the ***java.awt.event*** package. Here is that interface:

Interfaces and Callbacks

```
public interface ActionListener  
{  
    void actionPerformed(ActionEvent event);  
}
```

- ▶ The timer calls the actionPerformed method when the time interval has expired.

Interfaces and Callbacks

```
class TimePrinter implements ActionListener
{
    public void actionPerformed(ActionEvent event)
    {
        System.out.println("At the tone, the time is " +
            new Date());
        Toolkit.getDefaultToolkit().beep();
    }
}
```

Interfaces and Callbacks

In the needed place to make callbacks

```
ActionListener listener = new TimePrinter();  
Timer t = new Timer(10000, listener);
```

Functional Interfaces

- ❑ **Functional programming** is a style of programming (or a programming paradigm) where programs are executed by evaluating expressions.
- ❑ The output of a function is dependent on the values of its inputs [*This means that if we call a function x amount of times with the same parameters we'll get exactly the same result every time*].
- ❑ Allows us to focus on the problem rather than the code [*you specify what you want to accomplish in a task, but not how to accomplish it*]
- ❑ Functions can take functions as arguments and return functions as results.

Functional Interfaces

- ▶ Functional interfaces are new additions in **java 8** which permit exactly one abstract method inside them.
- ▶ These interfaces are also called **Single Abstract Method interfaces (SAM Interfaces)**.
- ▶ These can be represented using Lambda expressions, Method reference and constructor references as well.
- ▶ Java 8 introduces an annotation i.e. **@FunctionalInterface** too, which can be used for compiler level errors when the interface you have annotated violates the contracts of Functional Interface.

Functional Interfaces

```
package functionalInterfaceExample;  
  
@FunctionalInterface  
  
public interface MyFirstFunctionalInterface {  
    public void firstWork();  
  
}
```


Functional Interfaces

```
package functionalInterfaceExample;  
  
@FunctionalInterface  
public interface MyFirstFunctionalInterface {  
    public void firstWork();  
    public void doSomeMoreWork();  
}
```

Compilation Error

Unexpected @FunctionalInterface annotation
@FunctionalInterface MyFirstFunctionalInterface is not a functional interface
multiple non-overriding abstract methods found in interface
MyFirstFunctionalInterface

Functional Interfaces

```
package functionalInterfaceExample;  
  
@FunctionalInterface  
public interface MyFirstFunctionalInterface {  
    public void firstWork();  
    default public void doSomeMoreWork() {  
        .....  
    }  
}
```

Compilation Error

Unexpected @FunctionalInterface annotation
@FunctionalInterface MyFirstFunctionalInterface is not a functional interface
multiple non-overriding abstract methods found in interface
MyFirstFunctionalInterface

Generic Functional Interface

- The functional interface associated can be

```
// Use a generic functional interface.  
// A generic functional interface with two parameters  
// that returns a boolean result.  
@FunctionalInterface  
public interface SomeTest<T>  
{  
    boolean test(T n, T m);  
}
```

Generic Functional Interface

```
// Class SomeTestImpl
class SomeTestImpl implements SomeTest<Integer>
{
    public boolean test(Integer n, Integer m) {
        return n%m ==0;
    }
}
```

Generic Functional Interface

```
class GenericFunctionalInterfaceDemo
{
    public static void main(String args[]) {
        SomeTestImpl isFactor = new SomeTestImpl();

        if(isFactor.test(10, 2))
            System.out.println("2 is a factor of 10");

        System.out.println();
    }
}
```

Functional Interfaces Examples

- ▶ Package `java.util.function` contains several functional interfaces.
- ▶ Throughout the table, T and R are generic type names that represent the type of the object on which the functional interface operates and the return type of a method, respectively.
- ▶ The Following Tables show six of the basic generic functional interfaces.

Functional Interfaces Examples

Interface	Description
<u><i>BinaryOperator<T></i></u> “Represents an operation upon two operands of the same type, producing a result of the same type as the operands.”	Contains method apply that takes two T arguments, performs an operation on them (such as a calculation) and returns a value of type T.
<u><i>Consumer<T></i></u> “Represents an operation that accepts a single input argument and returns no result”	Contains method accept that takes a T argument and returns void. Performs a task with it's T argument, such as outputting the object, invoking a method of the object, etc.
<u><i>Function<T,R></i></u> “Represents a function that accepts one argument and produces a result.”	Contains method apply that takes a T argument and returns a value of type R. Calls a method on the T argument and returns that method's result.

28-Jan-25

56

Functional Interfaces Examples

Interface	Description
<u><i>Predicate<T></i></u> “Represents a predicate (boolean-valued function) of one argument.”	Contains method test that takes a T argument and returns a boolean. Tests whether the T argument satisfies a condition.
<u><i>Supplier<T></i></u> “Represents a supplier of results. There is no requirement that a new or distinct result be returned each time the supplier is invoked.”	Contains method get that takes no arguments and produces a value of type T. Often used to create a collection object in which a stream operation’s results are placed.
<u><i>UnaryOperator<T></i></u> “Represents an operation on a single operand that produces a result of the same type as its operand”	Contains method apply that takes an arguments of type T and returns a value of type T.

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes**
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Lesson 6

Wrapper Classes, Autoboxing and Inner Classes



Wrapper Classes

- Each primitive data type has a corresponding wrapper class.

boolean	→	Boolean
byte	→	Byte
char	→	Character
short	→	Short
int	→	Integer
long	→	Long
float	→	Float
double	→	Double

Wrapper Classes cont'd

- There are three reasons that you might use a wrapper class rather than a primitive:
 1. As an argument of a method that expects an object.
 2. To use constants defined by the class,
 - such as **MIN_VALUE** and **MAX_VALUE**, that provide the upper and lower bounds of the data type.
 3. To use class methods for
 - converting values to and from other primitive types,
 - converting to and from strings,
 - converting between number systems (decimal, octal, hexadecimal, binary).

Wrapper Classes cont'd

- They have useful methods that perform some general operation, for example:

<code>primitive xxxValue()</code>	→ convert wrapper object to primitive
-----------------------------------	---------------------------------------

<code>primitive parseXXX(String)</code>	→ convert String to primitive
---	-------------------------------

<code>Wrapper valueOf(String)</code>	→ convert String to Wrapper
--------------------------------------	-----------------------------

```
Integer i2 = new Integer(42);  
byte b = i2.byteValue();  
double d = i2.doubleValue();
```

```
String s3 =  
Integer.toHexString(254);  
System.out.println("254 is " + s3);
```

Wrapper Classes cont'd

- They have special static representations, for example:

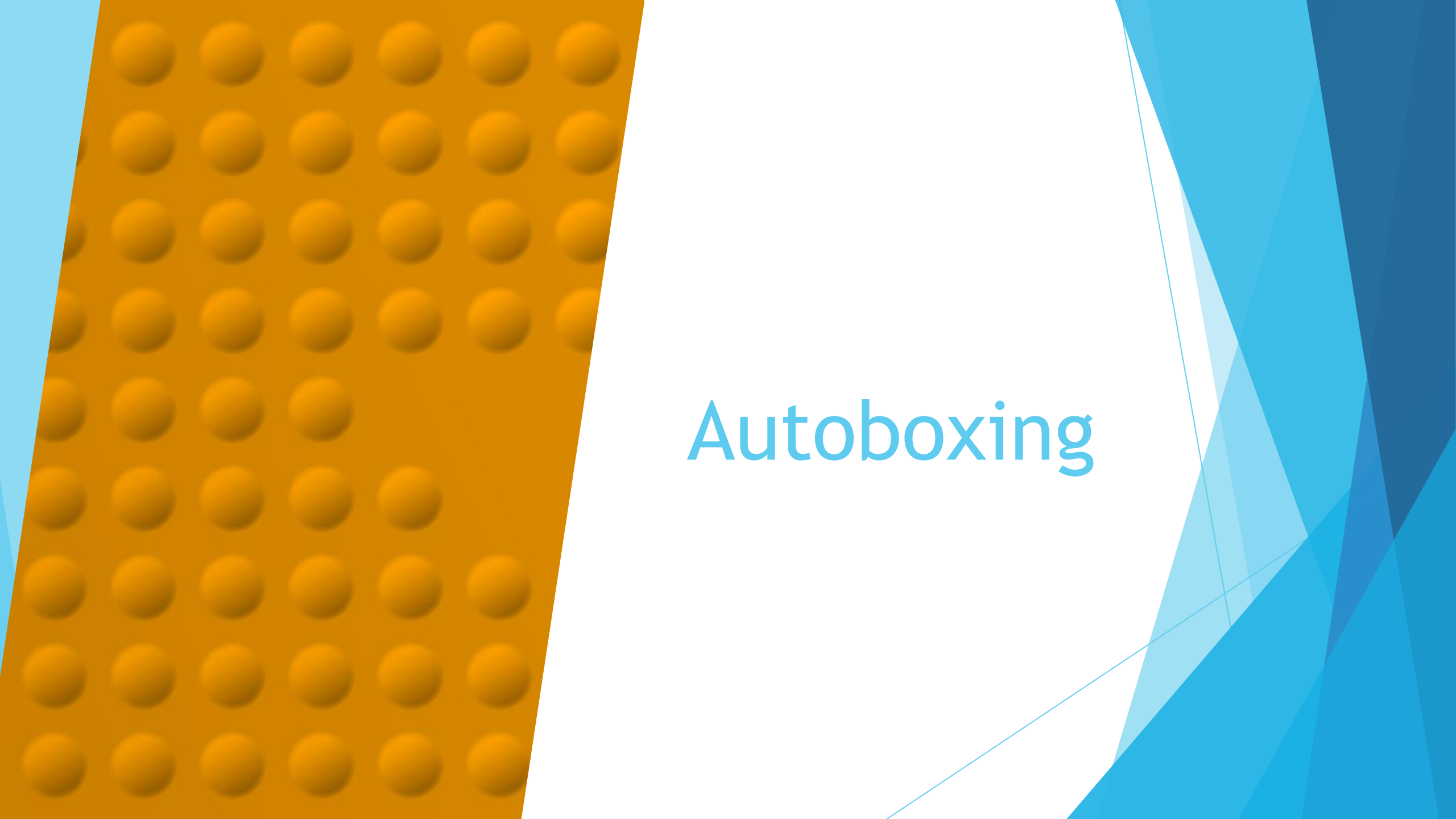
POSITIVE_INFINITY

NEGATIVE_INFINITY

NaN

Not a Number

In class Float &
Double

The background features a white central area. On the left, there is a large orange shape with a grid of circular indentations, resembling a LEGO brick. On the right, there are several overlapping blue and light blue geometric shapes, including triangles and polygons, creating a dynamic, abstract design.

Autoboxing

Autoboxing

- Beginning with JDK 5, Java has included two important features: *autoboxing* and *auto-unboxing*.
- **Autoboxing** is the process by which a primitive type is automatically encapsulated (boxed) into its equivalent type wrapper whenever an object of that type is needed.
- **Auto-unboxing** is the process by which the value of a boxed object is automatically extracted (unboxed) from a type wrapper when its value is needed.
- There is no need to call a method such as **intValue()** or **doubleValue()**.

Autoboxing

```
Integer intObject=100; //autobox an int  
int i = intObject; //auto-unbox
```

Autoboxing and Methods

- In addition to the simple case of assignments, autoboxing automatically occurs whenever a primitive type must be converted into an object; auto-unboxing takes place whenever an object must be converted into a primitive type.



Lab Exercises

Lab Exercise

- Develop an application to convert temperature from Celsius to Fahrenheit using Function<T,R>

```
public class TemperatureConvert implements Function<Float,Float>{  
  
    @Override  
    public Float apply(Float t) {  
  
        return                  ;  
  
    }  
  
}
```

```
public class TempMain {  
    public static void main(String[] args) {  
        float x=24;  
        System.out.println("Temp is = "+x+"C or "+  
            new TemperatureConvert().apply(t: x)+" F");  
    }  
}
```

Lab Exercise

- Use the interfaces in `java.util.function` to build an application that defines the roots of the quadratic equation $(ax^2 + bx + c = 0)$ and the roots could be computed by the following formula $(x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a})$
- <https://www.javatpoint.com/java-program-to-solve-quadratic-equation>